AMS 526: Numerical Linear Algebra for Computational and Data Sciences

Lecture 6: Floating Point Arithmetic; Accuracy and Stability

Xiangmin Jiao

Stony Brook University

Outline

1. **Floating Point Arithmetic (NLA §13)**
2. Accuracy and Stability (NLA §14–15)

Floating Point Representations

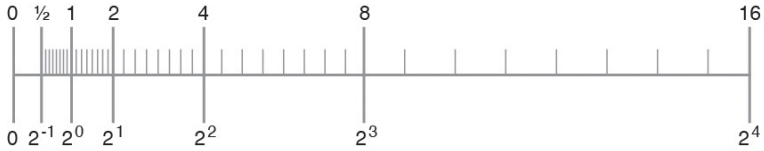
- Computers use a finite number of bits to represent real numbers.
  - Numbers cannot be arbitrarily large or small (associated risks of *overflow* and *underflow*).
  - There must be gaps between representable numbers (potential round-off errors).
  - Numerical linear algebra originated from analyzing the effect of rounding errors (von Neumann, Turing, Wilkinson, etc.).
- Commonly used computer representations are floating point representations, which resemble scientific notation:

$$\pm(d_0 + d_1\beta^{-1} + \cdots + d_{p-1}\beta^{-p+1})\beta^e, \quad 0 \le d_i < \beta,$$

  where β is the base, p is the number of digits of precision, and e is the exponent between $e_{\min}$ and $e_{\max}$.
- Normalize if $d_0 \neq 0$ (except for 0).

Floating Point Representations Cont'd

- We focus on binary numbers with $\beta = 2$.
- Gaps between adjacent numbers scale with the size of the numbers.



- Relative resolution given by *machine epsilon* $\epsilon_{\text{machine}} = 0.5\beta^{1-p}$.
- For all x (barring overflow and underflow), there exists a floating point x' such that $|x - x'| \leq \epsilon_{\text{machine}}|x|$.
- Note: The gap between 0 and the smallest number ($2^{e_{\min}}$) may be filled by subnormal numbers due to *denormalization*. For subnormal numbers, $|x - x'| > \epsilon_{\text{machine}}|x|$. We do not consider denormalization here.

IEEE Floating Point Representations

- Single precision: 32 bits.
  - 1 sign bit (S), 8 exponent bits (E), 23 significant bits (M):

$$(-1)^S \times 1.M \times 2^{E-127}.$$

  - $\epsilon_{\text{machine}}$ is $2^{-24} \approx 6 \times 10^{-8}$.
- Double precision: 64 bits.
  - 1 sign bit (S), 11 exponent bits (E), 52 significant bits (M):

$$(-1)^S \times 1.M \times 2^{E-1023}.$$

  - $\epsilon_{\text{machine}}$ is $2^{-53} \approx 1.11 \times 10^{-16}$.
- Modern architectures now also support half precision (16-bit) and even quarter precision (8-bit), motivated by machine-learning applications.
- Special quantities:
  - $+\infty$ and $-\infty$ when operation overflows; e.g., $x/0$ for nonzero x.
  - NaN (Not a Number) is returned when an operation has no well-defined result; e.g., 0/0, arcsin(2), NaN.

Floating Point Arithmetic

- Define $\mathrm{fl}(x)$ as the closest floating point approximation to x.
- By definition of $\epsilon_{\text{machine}}$, we have:

$$\text{For all } x \in \mathbb{R}, \text{ there exists } \epsilon \text{ with } |\epsilon| \leq \epsilon_{\text{machine}} \\ \text{such that } \mathrm{fl}(x) = x(1+\epsilon).$$

- Given operation $+$, $-$, $\times$, and $/$ (denoted by $*$), floating point numbers x and y, and corresponding floating point arithmetic (denoted by $\circledast$), we require that $x \circledast y = \mathrm{fl}(x * y)$.
- This is guaranteed by IEEE floating point arithmetic.
- Fundamental axiom of floating point arithmetic:

$$\text{For all } x, y \in \mathbb{F}, \text{ there exists } \epsilon \text{ with } |\epsilon| \leq \epsilon_{\text{machine}} \\ \text{such that } x \circledast y = (x * y)(1+\epsilon).$$

- These properties will be the basis of error analysis with rounding errors.
- Note that floating point arithmetic is **not associative**.

Outline

1. Floating Point Arithmetic (NLA §13)
2. **Accuracy and Stability (NLA §14–15)**

Accuracy

- Roughly speaking, accuracy means that "error" is small in an *asymptotic* sense, say $\mathcal{O}(\epsilon_{\text{machine}})$.
- Notation $\varphi(t) = \mathcal{O}(\psi(t))$ means $\exists C$ s.t. $|\varphi(t)| \leq C|\psi(t)|$ as t approaches 0 (or ∞).
  - Example: $\sin^2 t = \mathcal{O}(t^2)$ as $t \to 0$.
- If φ depends on s and t, then $\varphi(s,t) = \mathcal{O}(\psi(t))$ means $\exists C$ s.t. $|\varphi(s,t)| \leq C|\psi(t)|$ for any s as t approaches 0 (or ∞).
  - Example: $\sin^2 t \sin^2 s = \mathcal{O}(t^2)$ as $t \to 0$.
- When we say $\mathcal{O}(\epsilon_{\text{machine}})$, we are thinking of a series of idealized machines for which $\epsilon_{\text{machine}} \to 0$.

More on Accuracy

- An algorithm $\tilde{f}$ is *accurate* if the **relative** error is in the order of machine precision, i.e.,

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = \mathcal{O}(\epsilon_{\text{machine}}),$$

  i.e., $\leq C_1 \epsilon_{\text{machine}}$ as $\epsilon_{\text{machine}} \to 0$, where constant C_1 may depend on the condition number and the algorithm itself.
- In most cases, we expect

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = \mathcal{O}(\kappa \epsilon_{\text{machine}}),$$

  i.e., $\leq C \kappa \epsilon_{\text{machine}}$ as $\epsilon_{\text{machine}} \to 0$, where constant C should be independent of κ and the value of x (although it may depend on the dimension of x).
- How do we determine whether an algorithm is accurate or not?
  - It turns out to be an extremely subtle question.
  - A forward error analysis (operation by operation) is often too difficult and impractical, and cannot capture dependence on the condition number.
  - An effective solution is *backward error analysis*.

Stability

- We say an algorithm is *stable* if it gives "nearly the right answer to nearly the right question."
- More formally, an algorithm $\tilde{f}$ for problem f is *stable* if $(\forall x)$

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = \mathcal{O}(\epsilon_{\text{machine}})$$

for some $\tilde{x}$ with $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$.

Stability

- We say an algorithm is *stable* if it gives "nearly the right answer to nearly the right question."
- More formally, an algorithm $\tilde{f}$ for problem f is *stable* if $(\forall x)$

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = \mathcal{O}(\epsilon_{\text{machine}})$$

  for some $\tilde{x}$ with $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$.
- We say an algorithm is *backward stable* if it gives "exactly the right answer to nearly the right question."
- More formally, an algorithm $\tilde{f}$ for problem f is *backward stable* if $(\forall x)$

$$\tilde{f}(x) = f(\tilde{x})$$

  for some $\tilde{x}$ with $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$.

Stability

- We say an algorithm is *stable* if it gives "nearly the right answer to nearly the right question."
- More formally, an algorithm $\tilde{f}$ for problem f is *stable* if $(\forall x)$

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = \mathcal{O}(\epsilon_{\text{machine}})$$

  for some $\tilde{x}$ with $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$.
- We say an algorithm is *backward stable* if it gives "exactly the right answer to nearly the right question."
- More formally, an algorithm $\tilde{f}$ for problem f is *backward stable* if $(\forall x)$

$$\tilde{f}(x) = f(\tilde{x})$$

  for some $\tilde{x}$ with $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$.
- Is stability or backward stability stronger?

Stability

- We say an algorithm is *stable* if it gives "nearly the right answer to nearly the right question."
- More formally, an algorithm $\tilde{f}$ for problem f is *stable* if $(\forall x)$

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = \mathcal{O}(\epsilon_{\text{machine}})$$

  for some $\tilde{x}$ with $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$.
- We say an algorithm is *backward stable* if it gives "exactly the right answer to nearly the right question."
- More formally, an algorithm $\tilde{f}$ for problem f is *backward stable* if $(\forall x)$

$$\tilde{f}(x) = f(\tilde{x})$$

  for some $\tilde{x}$ with $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$.
- Is stability or backward stability stronger?
  - Backward stability is stronger.
- Does the (backward) stability of an algorithm depend on condition number of $f(x)$?

Stability

- We say an algorithm is *stable* if it gives "nearly the right answer to nearly the right question."
- More formally, an algorithm $\tilde{f}$ for problem f is *stable* if $(\forall x)$

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = \mathcal{O}(\epsilon_{\text{machine}})$$

  for some $\tilde{x}$ with $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$.
- We say an algorithm is *backward stable* if it gives "exactly the right answer to nearly the right question."
- More formally, an algorithm $\tilde{f}$ for problem f is *backward stable* if $(\forall x)$

$$\tilde{f}(x) = f(\tilde{x})$$

  for some $\tilde{x}$ with $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$.
- Is stability or backward stability stronger?
  - Backward stability is stronger.
- Does the (backward) stability of an algorithm depend on condition number of $f(x)$?
  - No.

Stability of Floating Point Arithmetic

- Backward stability of floating point operations is implied by these two floating point axioms:
  1. $\forall x \in \mathbb{R}$, $\exists \epsilon$, $|\epsilon| \leq \epsilon_{\text{machine}}$ s.t. $\text{fl}(x) = x(1+\epsilon)$.
  2. For floating-point numbers x, y, $\exists \epsilon$, $|\epsilon| \leq \epsilon_{\text{machine}}$ s.t. $x \circledast y = (x * y)(1+\epsilon)$.
- Example: Subtraction $f(x_1, x_2) = x_1 - x_2$ with floating-point operation

$$\tilde{f}(x_1, x_2) = \text{fl}(x_1) \ominus \text{fl}(x_2).$$

  - Axiom 1 implies $\text{fl}(x_1) = x_1(1+\epsilon_1)$, $\text{fl}(x_2) = x_2(1+\epsilon_2)$, for some $|\epsilon_1|, |\epsilon_2| \leq \epsilon_{\text{machine}}$.
  - Axiom 2 implies $\text{fl}(x_1) \ominus \text{fl}(x_2) = (\text{fl}(x_1) - \text{fl}(x_2))(1+\epsilon_3)$ for some $|\epsilon_3| \leq \epsilon_{\text{machine}}$.
  - Therefore,

$$\begin{aligned}
\text{fl}(x_1) \ominus \text{fl}(x_2) &= (x_1(1+\epsilon_1) - x_2(1+\epsilon_2))(1+\epsilon_3) \\
&= x_1(1+\epsilon_1)(1+\epsilon_3) - x_2(1+\epsilon_2)(1+\epsilon_3) \\
&= x_1(1+\epsilon_4) - x_2(1+\epsilon_5),
\end{aligned}$$

  where $|\epsilon_4|, |\epsilon_5| \leq 2\epsilon_{\text{machine}} + \mathcal{O}(\epsilon_{\text{machine}}^2)$.

Stability of Floating Point Arithmetic Cont'd

- Example: Inner product $f(x, y) = x^\top y$ using floating-point operations $\otimes$ and $\oplus$ is backward stable.
- Example: Outer product $f(x, y) = xy^\top$ using $\otimes$ and $\oplus$ is not backward stable.
- Example: $f(x) = x + 1$ computed as $\tilde{f}(x) = \mathrm{fl}(x) \oplus 1$ is not backward stable.
- Example: $f(x, y) = x + y$ computed as $\tilde{f}(x, y) = \mathrm{fl}(x) \oplus \mathrm{fl}(y)$ is backward stable.

Accuracy of Backward Stable Algorithm

Theorem

If a backward stable algorithm $\tilde{f}$ is used to solve a problem f with condition number κ using floating-point numbers satisfying the two axioms, then

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = \mathcal{O}(\kappa(x)\epsilon_{\text{machine}}).$$

Proof

Backward stability means $\tilde{f}(x) = f(\tilde{x})$ for $\tilde{x}$ such that $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$. The definition of the condition number gives

$$\frac{\|f(\tilde{x}) - f(x)\|}{\|f(x)\|} \leq (\kappa(x) + o(1)) \frac{\|\tilde{x} - x\|}{\|x\|},$$

where $o(1) \to 0$ as $\epsilon_{\text{machine}} \to 0$. Combining the two gives the desired result.

Accuracy of Backward Stable Algorithm

Theorem

If a backward stable algorithm $\tilde{f}$ is used to solve a problem f with condition number κ using floating-point numbers satisfying the two axioms, then

$$\frac{\|\tilde{f}(x) - f(x)\|}{\|f(x)\|} = \mathcal{O}(\kappa(x)\epsilon_{\text{machine}}).$$

Proof

Backward stability means $\tilde{f}(x) = f(\tilde{x})$ for $\tilde{x}$ such that $\|\tilde{x} - x\|/\|x\| = \mathcal{O}(\epsilon_{\text{machine}})$. The definition of the condition number gives

$$\frac{\|f(\tilde{x}) - f(x)\|}{\|f(x)\|} \leq (\kappa(x) + o(1))\frac{\|\tilde{x} - x\|}{\|x\|},$$

where $o(1) \to 0$ as $\epsilon_{\text{machine}} \to 0$. Combining the two gives the desired result.